

Black-Box Prompt Optimisation — State of the Field

A survey of automatic prompt optimisation (APO) for closed-weight LLMs, focused on 2023–2026 research. Includes a method comparison and the unresolved challenges that define the next research wave.

TL;DR (one screen)

The problem. You have access to an LLM only through an API. You cannot touch its weights. You want the LLM to behave better on a task. The only handle you have is the *prompt*. Black-box prompt optimisation (BPO, also called APO — automatic prompt optimisation) is the field of automated methods that search the prompt space using only API outputs and a scoring function.

The current state in mid-2026. The field has matured from "LLMs can write their own prompts" (APE, 2022) through evolutionary search (EvoPrompt, PromptBreeder), LLM-as-optimiser (OPRO), textual-gradient methods (ProTeGi, TextGrad), and Bayesian optimisation of joint instruction + demonstrations (MIPROv2). The current frontier is **reflective evolution** — GEPA (ICLR 2026 Oral) shows that an optimiser which reads full execution traces and writes natural-language diagnoses can outperform reinforcement learning (GRPO) by 6–20% while using 35× fewer rollouts, and beats MIPROv2 by 10%+.

Why it matters for agent memory. Memory-management systems are now almost entirely prompt-driven (Generative Agents, A-Mem, Hindsight, MemR3). The prompts that control importance scoring, retrieval ranking, and reflection are hand-tuned. No published work has applied modern APO to memory-management prompts specifically. That is the research gap.

1. Problem formulation

Given:

- A target LLM M accessible only through a forward-only API.
- A task distribution with input–output examples and a scoring function $s(M(p, x), y)$ that takes a model output and ground truth and returns a scalar.
- An optional language model M_{opt} (often the same model, sometimes a smaller/cheaper one) used as the optimiser.

Find:

- A prompt p^* (instruction text, optionally with few-shot demonstrations) that maximises expected score over the task distribution.

Constraints unique to the black-box setting:

- No access to weights, gradients, logits, or intermediate activations of M .
- Every evaluation costs an API call.
- Outputs are stochastic — same prompt scored twice will differ.

This is a **discrete, high-dimensional, noisy, expensive black-box optimisation problem** over a space of natural-language strings.

2. Five-stage unifying framework

The 2025 systematic survey (Vatsal et al., arXiv 2502.16923) decomposes every APO method into five stages. Useful for comparison:

Stage	What it does	Design choice examples
1. Seed initialisation	Pick the starting prompt(s)	Human-written, LLM-generated, retrieved from a library
2. Inference + evaluation	Run the candidate, score it	Exact match, LLM-as-judge, multi-metric, full execution trace
3. Feedback generation	Convert scores into editing signal	Scalar reward, textual critique, "gradient" feedback
4. Candidate proposal	Generate the next prompt(s)	LLM-rewrite, evolutionary mutation, beam expansion, Bayesian acquisition
5. Filter + retain	Decide which prompts survive	Top-k, Pareto-front, successive halving, tournament

Different methods make different choices at each stage. The recent gains in 2024–2026 come almost entirely from richer choices at stages 3 (feedback) and 5 (selection), not from cleverer mutation operators.

3. Method families

Family A — Sampling + selection

APE — Automatic Prompt Engineer (Zhou et al., 2022, <https://arxiv.org/abs/2211.01910>)

- Uses an LLM to *propose* a batch of candidate instructions from a small set of input–output examples.
- Scores each candidate on a validation set.
- Keeps the best.
- First convincing demonstration that LLM-generated prompts can match human-engineered ones.
- Weakness: no iteration — one round of sampling, no learning across rounds.

Family B — Evolutionary / population-based

EvoPrompt (Guo et al., 2023, <https://arxiv.org/abs/2309.08532>)

- Classical genetic algorithm: population of prompts, crossover, mutation, selection.
- Crossover and mutation are performed by an LLM via prompting.

- Outperforms APE on instruction-induction and BIG-Bench.

PromptBreeder (Fernando et al., DeepMind 2023, <https://arxiv.org/abs/2309.16797>)

- Self-referential: the *mutation prompts* themselves are also evolved.
- Diversity-maintenance heuristics prevent population collapse.
- Outperforms EvoPrompt on standard benchmarks.
- Weakness: very expensive — many LLM calls per generation.

Family C — LLM-as-optimiser

OPRO — Optimization by PROMpting (Yang et al., DeepMind 2023, <https://arxiv.org/abs/2309.03409>)

- At each step, the LLM is shown a *trajectory* of previous (prompt, score) pairs and asked to propose a better prompt.
- The LLM itself plays the role of the optimiser.
- Effective on small-scale optimisation (math word problems, classification).
- Insight: scoring history is enough context for an LLM to do credit assignment without explicit gradients.
- Weakness: scales poorly — once the trajectory gets long, the LLM cannot reason over it.

Family D — Textual gradients

APO / ProTeGi (Pryzant et al., 2023, <https://arxiv.org/abs/2305.03495>)

- Introduces "textual gradients": when a prompt fails on an example, the LLM is asked to write a natural-language critique of *why* it failed.
- That critique acts like a gradient signal pointing toward improvement.
- Beam search over critique-guided edits.

TextGrad (Yuksekgonul et al., Nature 2024, <https://arxiv.org/abs/2406.07496>)

- Formalises textual gradients into a full PyTorch-style framework.
- Backpropagates LLM-generated feedback through arbitrary computation graphs of LLM calls.
- Publishes the abstraction `loss.backward()` for text.
- Applications shown beyond prompts: drug-like molecule design, radiotherapy plans, code generation.
- This is the most theoretically-clean BPO framework to date.

REVOLVE (2024, <https://arxiv.org/abs/2412.03092>)

- Adds second-order optimisation: tracks how responses *change* across iterations, not just the gradient at one step.
- 29% improvement over TextGrad on code-optimisation tasks.

Family E — Bayesian / structured frameworks

DSPy + MIPROv2 (Khattab et al., Stanford NLP, <https://dspy.ai>)

- DSPy reframes prompting as Python programs with typed Signatures and Modules.

- MIPROv2 is the workhorse optimiser:

1. **Bootstrap** few-shot demonstrations from training data using a teacher LLM. 2. **Propose** candidate instruction strings via a "grounded proposer" that conditions on task metadata. 3. **Search** over (instruction × demo set) combinations using Bayesian optimisation (Optuna TPE).

- Strong on multi-module agent programs where multiple prompts are optimised jointly.
- The current production-grade default for serious teams.

BPO — Black-Box Prompt Optimization (Cheng et al., ACL 2024, <https://arxiv.org/abs/2311.04155>)

- A different approach: train a small open-source *prompt rewriter* model on a curated dataset of (bad prompt, good prompt) pairs.
- At inference time, every user prompt is rewritten before being sent to the target LLM.
- Reports 8.8–22.0% win-rate gains across GPT-3.5, GPT-4, Claude-2, Llama-2-chat, Vicuna.
- Strength: the optimiser is portable across target models.

Family F — Reflective evolution (the 2025–2026 frontier)

GEPA — Genetic-Pareto (Agrawal et al., ICLR 2026 Oral, <https://arxiv.org/abs/2507.19457>)

- Combines three ideas that earlier methods had separately:

1. **Full-trace reflection.** The optimiser reads not just the final answer and score, but the *entire execution trace* — reasoning chain, tool calls, error messages — and writes a natural-language diagnosis of what went wrong. 2. **Pareto-front maintenance.** Rather than tracking the single best prompt, GEPA maintains a Pareto front of prompts that each dominate on different instances. This prevents the optimiser from converging to a local optimum that overfits the validation set. 3. **Evolutionary recombination.** Prompts on the Pareto front are mutated and recombined via the reflective LLM.

- Empirical results:

- **Beats GRPO (reinforcement learning baseline) by 6–20%** while using **35× fewer rollouts**. - **Beats MIPROv2 by 10%+** on standard agent benchmarks.

- Available in DSPy as `dspy.GEPA` since late 2025.

SICA — Self-Improving Coding Agent (2025, <https://arxiv.org/abs/2512.09108>)

- Agents that edit their own source prompts and heuristics via LLM-proposed modifications.
- 17–53% improvement on coding tasks.
- Less general than GEPA but a useful proof point that optimisation can run *while the agent is in production*.

AutoPDL (2025, <https://arxiv.org/abs/2504.04365>)

- AutoML-style successive-halving over the combinatorial space of prompting *patterns* (CoT, ReAct, plan-and-solve, etc.) and demonstrations.
 - 9–69pp accuracy gains across tasks and models.
 - Useful when you do not yet know which prompting pattern is right for your task.
-

4. Comparison table

Method	Year	Family	Optimises	Feedback signal	Search	Best for	Limitation
APE	2022	Sample+select	Instructions	Scalar score	One-shot sampling	Quick baseline	No iteration
EvoPrompt	2023	Evolutionary	Instructions	Scalar score	GA	Discrete tasks	Many calls
PromptBreeder	2023	Evolutionary	Instructions + mutation prompts	Scalar score	GA + self-reference	Long-horizon search	Very expensive
ProTeGi (APO)	2023	Textual gradient	Instructions	Per-example critique	Beam	Tasks with clear failure modes	Sensitive to critic LLM
OPRO	2023	LLM-as-optimiser	Instructions	(prompt, score) history	LLM rewrite	Small-scale, fast iteration	Trajectory ceiling
BPO	2024	Trained rewriter	User prompts	Win-rate against base	Supervised fine-tune	Cross-model reuse	Needs labelled pairs
TextGrad	2024	Textual gradient	Anything	Textual gradient through graph	Backprop-style	Compound systems	Heavy infra
REVOLVE	2024	Textual gradient (2nd order)	Anything	Response-evolution	Backprop-style	Iterative refinement tasks	New, less validated
MIPROv2	2024	Bayesian + bootstrap	Instructions + demos	Scalar score	Bayesian opt (TPE)	Production agent programs	Cold-start cost
AutoPDL	2025	AutoML	Instructions + pattern + demos	Scalar score	Successive halving	Unknown best pattern	Limited to fixed pattern library
GEPA	2026	Reflective evolution	Instructions (+ demos)	Full trace + Pareto	Reflective GA	Multi-step agents, sample-limited	Newer; ecosystem maturing
SICA	2025	Self-editing	Live agent prompts	Online task reward	LLM self-edit	Production agents	Risk of drift

Three observations from this table:

1. The trend over time is toward **richer feedback signals** — from scalar to per-example critiques to full execution traces.
2. The state-of-the-art now requires **explicit anti-overfitting machinery** — Pareto fronts (GEPA), Bayesian priors (MIPROv2), or population diversity (PromptBreeder).
3. The compute cost is roughly *inversely proportional* to feedback richness: GEPA needs 35× fewer rollouts than RL precisely because each rollout produces a richer learning signal.

5. Surveys to anchor further reading

Survey	arXiv	Angle	Best feature
Vatsal et al.	https://arxiv.org/abs/2502.16923	Systematic, taxonomy-first	5-stage unifying framework used in §2
Wan et al.	https://arxiv.org/abs/2502.18746	Heuristic-search angle	Detailed comparison of search algorithms
Optimisation-theoretic	https://arxiv.org/abs/2502.11560	Formal optimisation lens	Variables × objectives × methods taxonomy

Read Vatsal first for breadth, then the optimisation-theoretic one if you want to ground the field in classical optimisation language.

6. Unresolved challenges

These come both from the explicit "future work" sections of the surveys and from limitations identified across the empirical papers.

6.1 Sample efficiency in genuinely closed settings

Every API call is real money. MIPROv2 routinely needs hundreds to thousands of evaluations per optimisation run on a non-trivial agent. GEPA cuts this dramatically with reflective feedback, but is still expensive for production deployments. *No method yet works robustly in the <100-evaluations regime that is realistic for cost-constrained users.*

6.2 Evaluation noise

LLM outputs are stochastic. The same prompt scored on the same example gives different results across runs. Most optimisers treat the score as deterministic. Methods that explicitly model evaluation noise (e.g. Bayesian optimisation in MIPROv2) help but do not solve the problem — and noise compounds when the scorer is itself an LLM-as-judge.

6.3 Validation-set overfitting

A prompt that scores well on a small held-out set often does not generalise. This is the prompt-engineering equivalent of overfitting in machine learning. GEPA's Pareto front partly addresses this. There is no consensus on how to construct or size validation sets for prompt optimisation.

6.4 Transferability across models

A prompt optimised for GPT-4 frequently degrades on Claude or Gemini. Each model has different sensitivities (formatting, ordering, role tokens). BPO partly addresses this with cross-model training, but most empirical work fixes one target model. *Cross-model prompt portability is an open research question with practical urgency as enterprises multi-source LLM providers.*

6.5 Optimisation of compound / agentic systems

Modern agents have many prompts (planner, tool selector, memory writer, memory reader, reflector). They interact non-linearly — a better planner prompt can be hurt by a worse memory reader. MIPROv2 and TextGrad can optimise multi-prompt programs, but the search space grows combinatorially, and there is no principled credit assignment when an agent fails ten steps deep into a trajectory.

6.6 Online / continual optimisation

Almost all published APO methods are offline — collect data, optimise, deploy. But deployed agents face distribution shift; the optimised prompt slowly degrades. SICA shows online self-editing is possible but risks drift and reward hacking. *Online prompt optimisation with stability guarantees is essentially uncharted.*

6.7 Evaluation of optimisers themselves

The field has no standardised benchmark. Different papers use different tasks (BIG-Bench, MMLU, HotpotQA, GSM8K, custom agent suites), different starting prompts, and different compute budgets. Comparisons across papers are weak. A shared APO-bench is overdue.

6.8 Lack of theoretical foundation

Most methods are empirical recipes. There are convergence guarantees for classical Bayesian optimisation and genetic algorithms, but none of these extend cleanly to the LLM-mediated discrete optimisation regime. The optimisation-theoretic survey (2502.11560) is the closest existing attempt at unifying theory, and it is mostly taxonomic, not predictive.

6.9 Open-ended tasks without clean metrics

APO assumes a scoring function. For open-ended creative or strategic tasks (writing, planning, social judgment), the scoring function is itself a hard problem. Current systems mostly fall back to LLM-as-judge, which inherits the same biases as the target LLM and creates circular optimisation pressure.

6.10 Safety and adversarial robustness

An optimised prompt can be very brittle in adversarial inputs even while scoring well on a clean validation set. Prompt sensitivity minimisation (e.g. PSM, <https://arxiv.org/abs/2511.16209>) is a new line of work but has not yet been integrated with mainstream APO.

7. Practical recommendations

For someone starting fresh on a single agent application

1. Start with **DSPy + MIPROv2**. Industry-grade, well-supported, joint instruction + few-shot optimisation, works for compound systems.
2. If MIPROv2 plateaus or you have a multi-step agent: switch to **GEPA** (also in DSPy now). Especially worth it if your task gives rich execution traces (tool calls, intermediate outputs).
3. Keep **TextGrad** in mind if you have a non-prompt component to optimise (a code template, a data extraction schema, etc.).
4. Avoid hand-rolling APE / EvoPrompt unless you specifically need to. Modern frameworks dominate them.

For applying APO to memory-management prompts (the agent-memory gap)

There is no published work doing this. A sensible research path:

1. Pick a concrete memory benchmark (LongMemEval is a good choice — multiple capabilities, clean scoring).
2. Wrap the Generative Agents importance-scoring prompt in a DSPy module.
3. Optimise with MIPROv2 first as a baseline, then GEPA.
4. Critical measurement: does the optimised importance-scoring prompt **transfer** between domains (social chat → task agent → coding), or does it overfit? If it overfits, that is a *general* problem with the LLM-as-memory-manager paradigm, not a quirk of the optimiser.

What to read in order if you have a week

1. **Day 1:** Vatsal survey (2502.16923) — get the taxonomy in your head.
2. **Day 2:** APE paper + OPRO paper — the two foundational ideas.
3. **Day 3:** TextGrad paper + the GitHub repo — the cleanest framework.
4. **Day 4:** MIPROv2 docs + DSPy tutorial — get hands-on.
5. **Day 5:** GEPA paper + dspy.GEPA implementation — see the current frontier.
6. **Day 6:** Run an experiment on a benchmark task with MIPROv2 vs GEPA.
7. **Day 7:** Pick an unresolved challenge from §6 and write down your hypothesis for what the next paper should be.

8. Bibliography (cited sources)

- APE — <https://arxiv.org/abs/2211.01910>
- ProTeGi — <https://arxiv.org/abs/2305.03495>
- OPRO — <https://arxiv.org/abs/2309.03409>
- EvoPrompt — <https://arxiv.org/abs/2309.08532>
- PromptBreeder — <https://arxiv.org/abs/2309.16797>
- BPO — <https://arxiv.org/abs/2311.04155>

- TextGrad — <https://arxiv.org/abs/2406.07496> (Nature 2024)
- REVOLVE — <https://arxiv.org/abs/2412.03092>
- MIPROv2 — <https://dspace.ai/api/optimizers/MIPROv2/>
- DSPy framework — <https://github.com/stanfordnlp/dspy>
- AutoPDL — <https://arxiv.org/abs/2504.04365>
- GEPA — <https://arxiv.org/abs/2507.19457> (ICLR 2026 Oral)
- SICA — <https://arxiv.org/abs/2512.09108>
- PSM — <https://arxiv.org/abs/2511.16209>
- Survey: Vatsal et al. — <https://arxiv.org/abs/2502.16923>
- Survey: Wan et al. — <https://arxiv.org/abs/2502.18746>
- Survey: Optimisation-theoretic — <https://arxiv.org/abs/2502.11560>